Contact Manager-Connect Hub

NM- Backend development by MongoDB

By:

Miss. Harini S(Reg.no.: 2413371058009)

Miss. Bhuvaneshwari K(Reg.no.: 2413371058005)

Miss. Durgailakshmi P (Reg.no.: 2413371058006)

Miss. Gayathri M(Reg.no.: 2413371058008)

II Bsc

Department of computer science

Quaid-E-Millath Government College for Women(Autonomous)

Chennai -02

March 2026

TABLE OF CONTENTS:

Introduction

The ConnectHub Backend API is a contact management system that allows users to securely store and manage contact information.

The system allows users to:

- Register a new account
- Login securely
- Add contacts
- View contacts
- Update contact details
- Delete contacts

The backend is developed using:

- Node.js
- Express.js

The application uses:

- MongoDB database
- Hosted on MongoDB Atlas

All APIs were tested using:

- Thunder Client inside Visual Studio Code.

Project Overview

ConnectHub is designed as a RESTful backend service. The system enables secure contact management through API endpoints.

The back end server receives requests from a client application and interacts with the MongoDB database to perform required operations.

Objectives Of The project

The main objectives of this project are:

- To design and implement a RESTful backend API.
- To manage user authentication.
- To perform CRUD operations on contacts.
- To integrate a cloud database.
- To test API functionality using Thunder Client

Scope Of The Project

The project focuses mainly on backend development and database integration.

Future improvements can include:

- Mobile application integrete
- Web interface using React
- Authentication using JWT tokens
- Advanced contact search

Technology Stack

Technology	Purpose
Node.js	Backend runtime
Express.js	Web framework
MongoDB	Database
MongoDB Atlas	Cloud hosting
Thunder Client	API testing
Visual studio code	Development environment

System Requirements

Hardware Requirements

- Processor: Intel i3 or higher
- RAM: 4GB or higher
- Hard Disk: 10GB free space

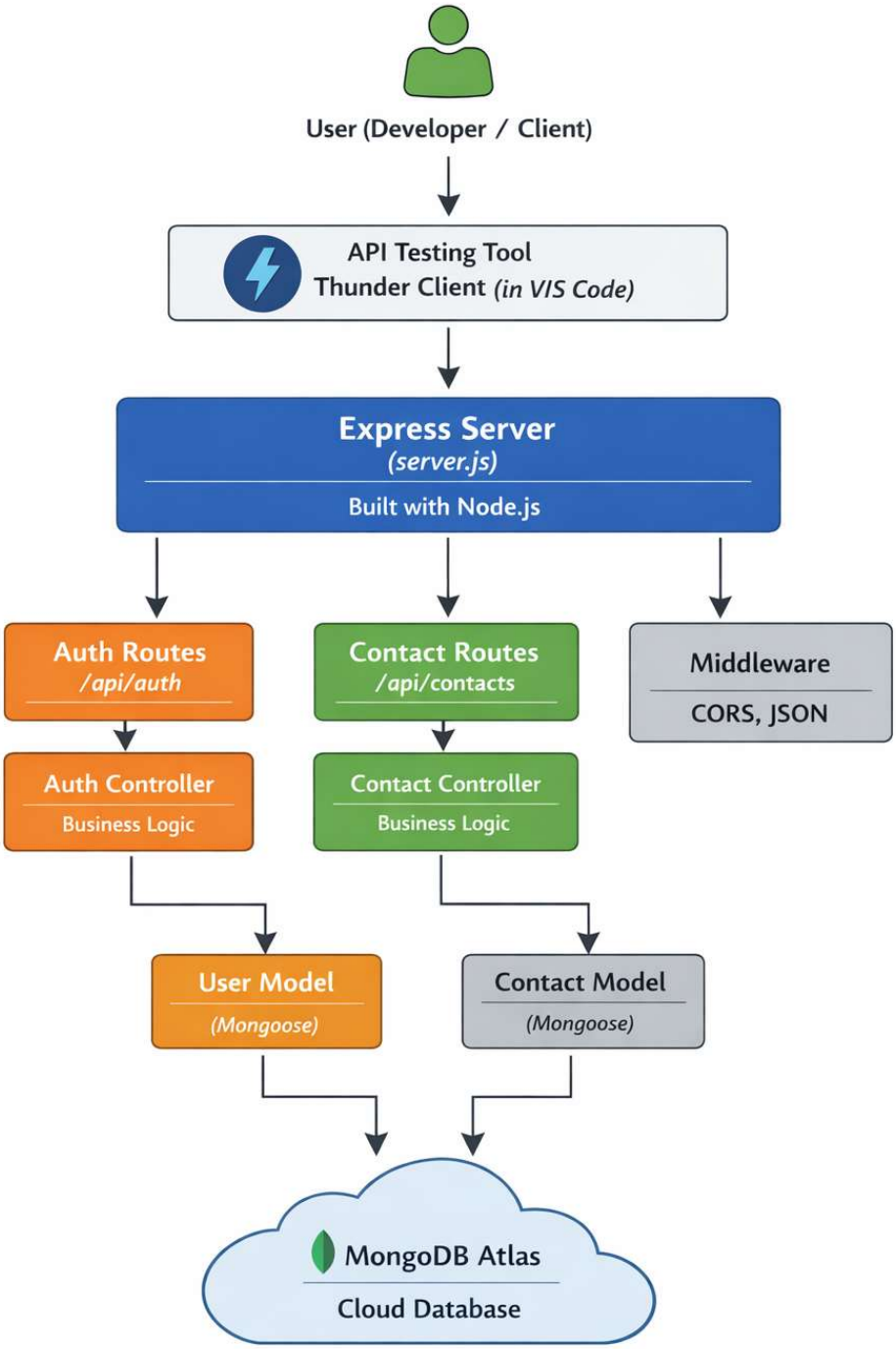
Software Requirements

- Node.js
- MongoDB Atlas Account
- Visual Studio Code
- Thunder Client Extension

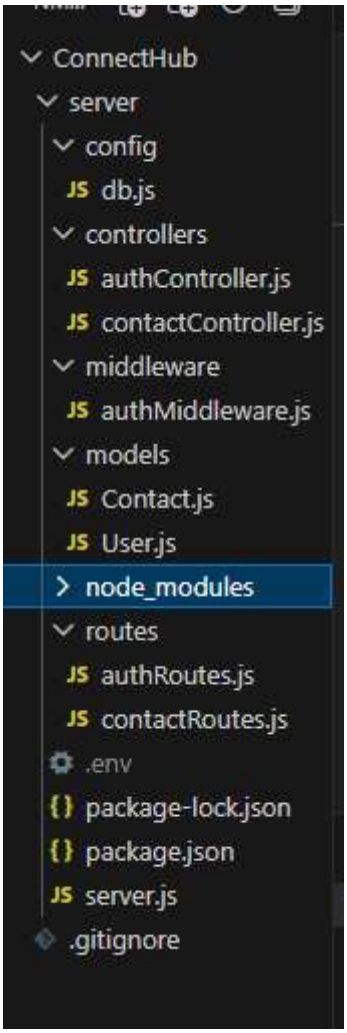
System Architecture

The application follows a client-server architecture.

Client tools send requests to the server. The server processes the requests and communicates with the MongoDB database.


User (Developer / Client)
API Testing Tool
Thunder Client (in VIS Code)
Express Server
(server.js)
Built with Node.js
Auth Routes
/api/auth
Contact Routes
/api/contacts
Middleware
CORS, JSON
Auth Controller
Business Logic
Contact Controller
Business Logic
User Model
(Mongoose)
Contact Model
(Mongoose)
MongoDB Atlas
Cloud Database

Project Structure



Database Design

The database consists of two main collections:

User Collection:

Field	Type
name	String
email	String
password	String

Contact Collection

Field	Type
name	String
phone	String
email	String
userId	ObjectId

System Flow Diagram

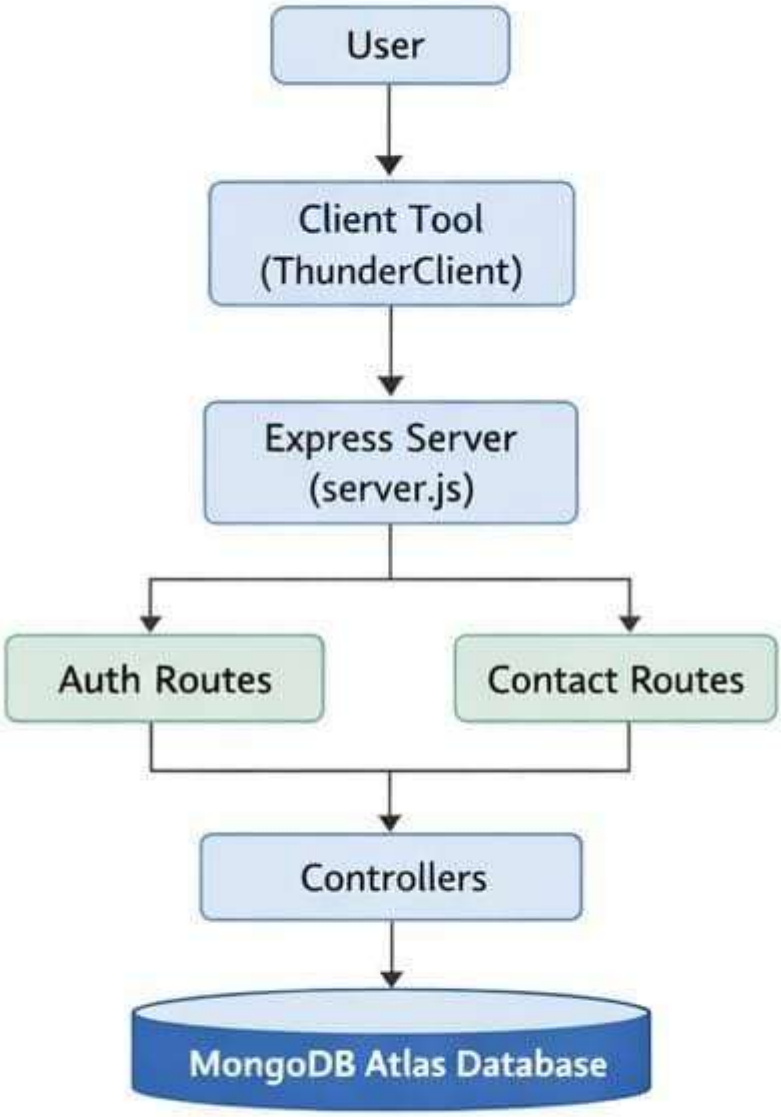


Figure: System Flow Diagram of Contact Manager

API Flow Diagram

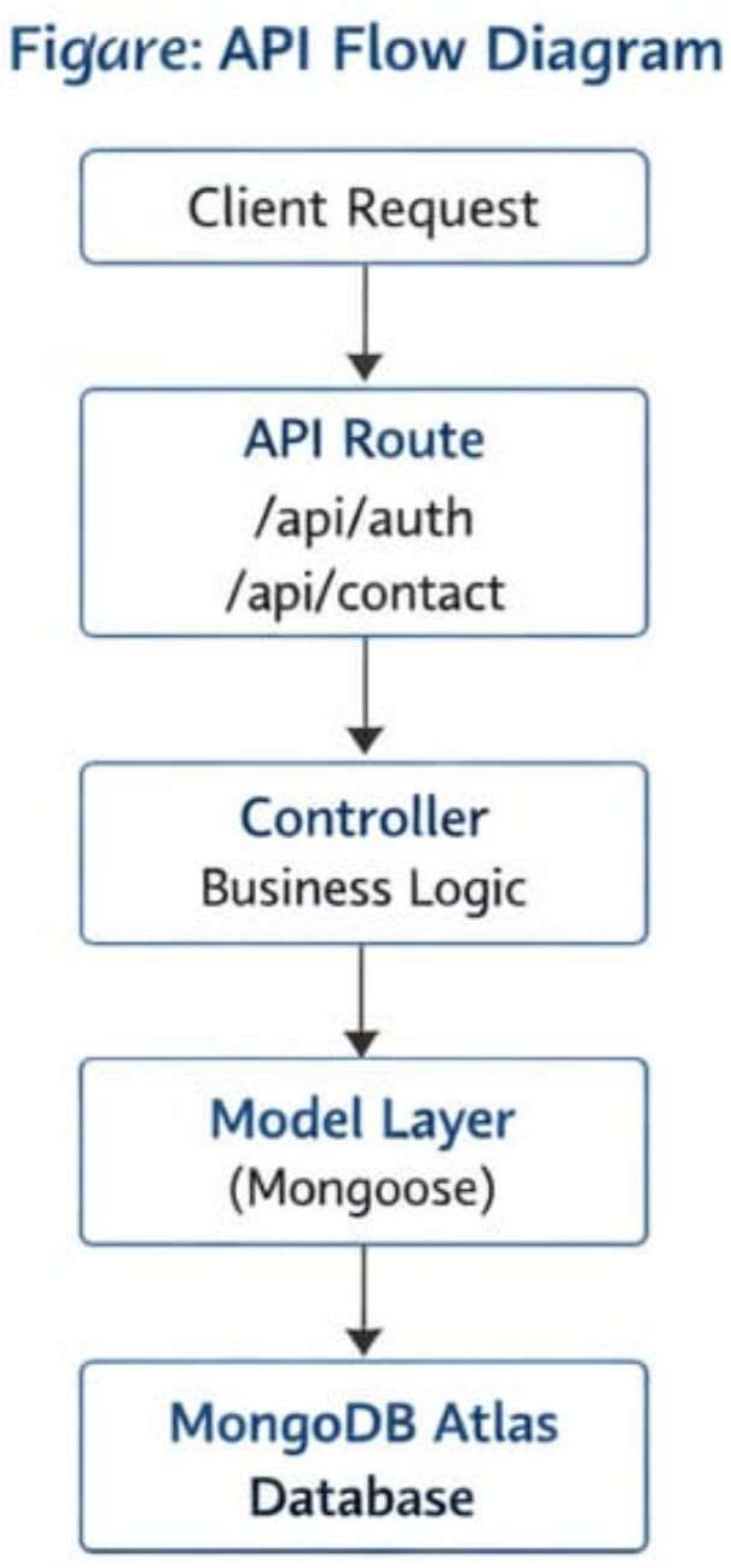
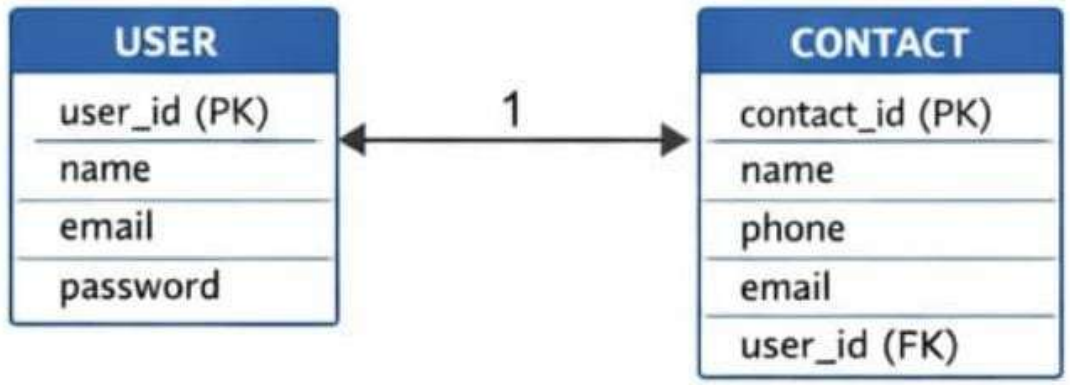


Figure: API Flow Diagram

Entity Relationship Diagram

Figure: Entity Relationship Diagram



Backend Implementation

The backend is implemented using Express.js.

Main components include:

- Server configuration
- Database connection
- Routes
- Controllers
- Models

The application uses REST API architecture to perform operations.

API Documentation

The application contains six APIs.

API 1 – Register User

Endpoint

POST /api/auth/register

Description

This API allows a new user to register in the system.

Request Body

{

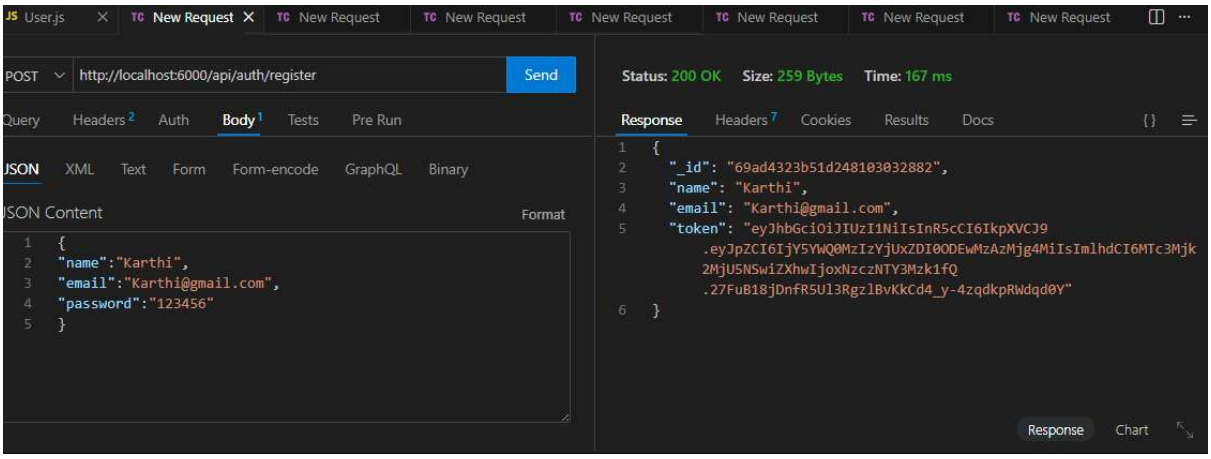
"name": "John",

"email": "john@example.com",

"password": "123456"

}

Screen shot



API-2 Login User

Endpoint

POST /api/auth/login

Description

This API authenticates a user and allows them to access the system.

Request Body

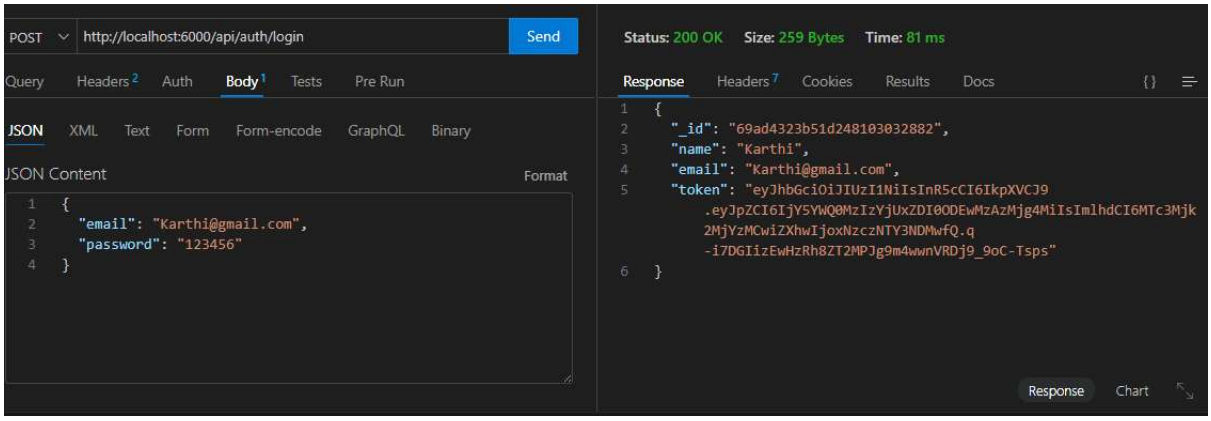
{

"email": "john@example.com",

"password": "123456"

}

Screen shot:



API 3 Add contact

Endpoint

POST /api/contacts

Description

This API adds a new contact to the database.

Request Body

{

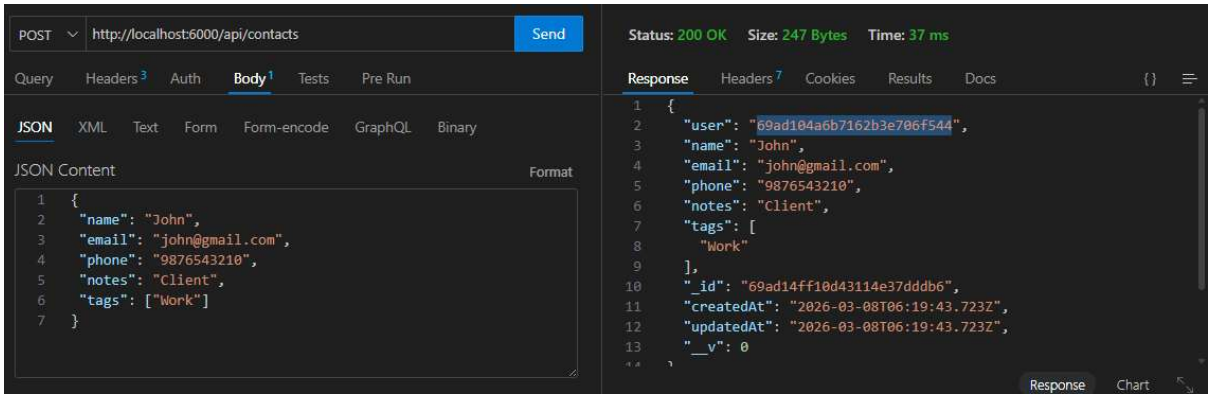
"name": "Alex",

"phone": "9876543210",

"email": "alex@example.com"

}

SCREEN SHOT



API 4 Get All Contacts

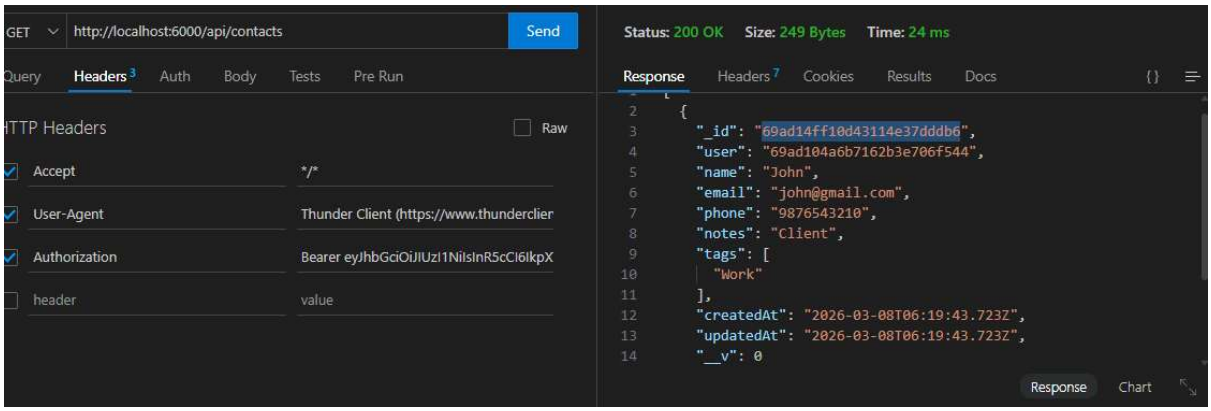
Endpoint

GET /api/contacts

Description

This API retrieves all stored contacts from the database.

SCREEN SHOT:



API 5 Update Contact

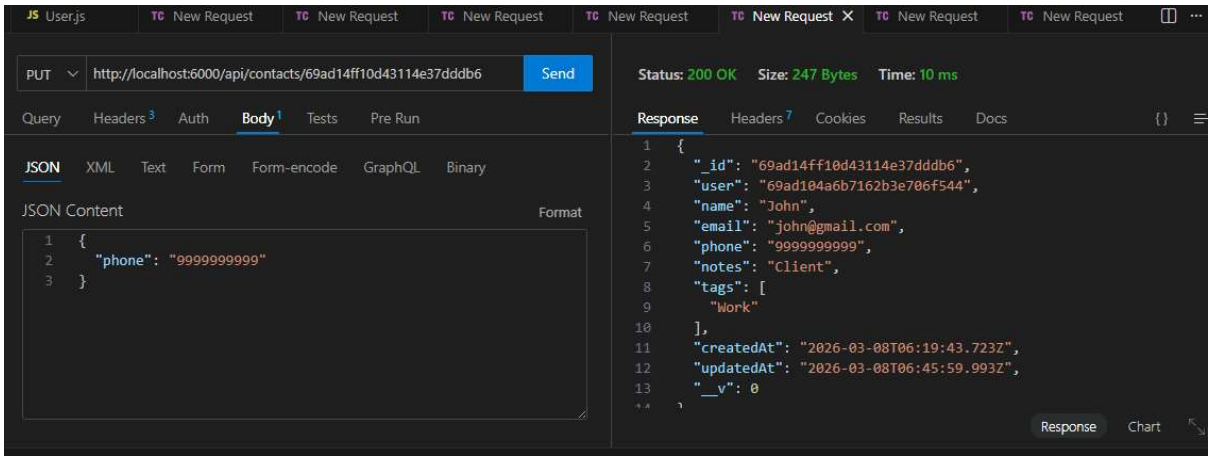
Endpoint

PUT /api/contacts/:id

Description

This API updates contact information.

SCREEN SHOT:



API 6 Delete Contact

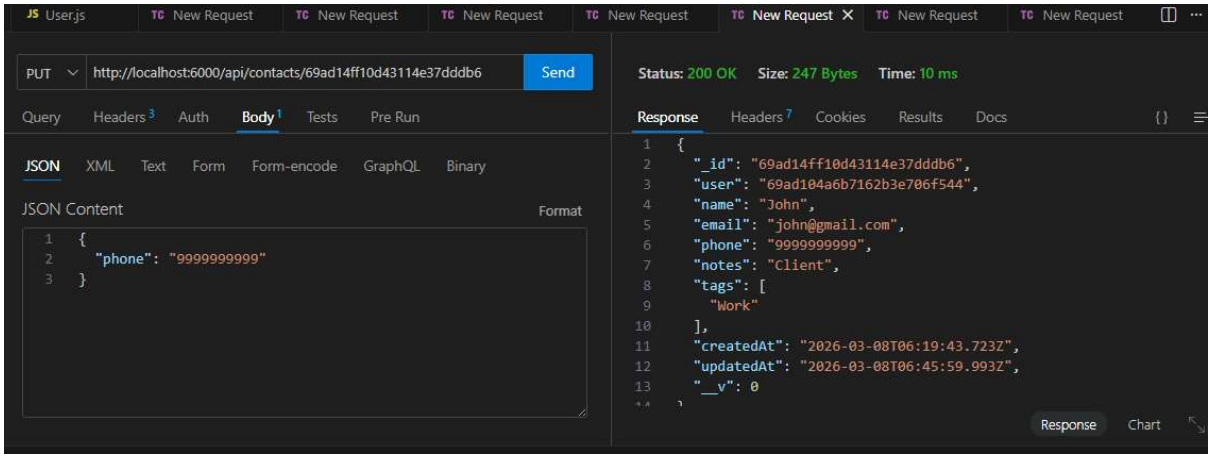
Endpoint

DELETE /api/contacts/:id

Description

This API deletes the selected contact.

SCREEN SHOT:



API Testing Using Thunder Client

All APIs were tested using Thunder Client extension in Visual Studio Code.

Testing involved:

- Sending HTTP requests
- Providing JSON request bodies
- Checking responses
- Verifying database changes

Security Considerations

The project uses the following security practices:

- Environment variables for sensitive data
- Secure MongoDB connection
- CORS protection
- Input validation
-

Challenges Faced

During development, some challenges included:

- MongoDB connection errors
- API route configuration
- Environment variable setup
- Debugging API responses

Future Enhancements

Possible improvements include:

- React frontend interface
- JWT authentication
- Search and filter contacts
- Pagination
- Mobile application integration

Conclusion

The ConnectHub backend API successfully demonstrates the development of a RESTful backend service using Node.js, Express.js, and MongoDB.

The system allows users to securely manage their contacts and perform CRUD operations efficiently.

References

Node.js Documentation

Express.js Documentation

MongoDB Atlas Documentation